BeyondNN: Auditing Interpretability Claims about Neural Networks with Provenance, Controls, and Counterevidence

Nikolaos Roufas and Andreas Kanavos

Abstract—Interpretability methods produce measurements, but a measurement does not by itself establish the claim commonly attached to it. We present BeyondNN, an evidence and audit layer for PyTorch that records typed, provenance-bound evidence, represents claims about neural networks by relation, subject, scope and eligible units, and re-derives each claim's standing under declared primary, alternative and stress_test configurations. We give a formal semantics for evidence admissibility, test outcomes and standings, with six properties that hold by construction. On six held-out sites of trained MLP, CNN, BERT-tiny and BERT-base models, 47 of 52 primary-supported Integrated Gradients necessity claims had at least one contradicting declared alternative; an exact re-execution of the frozen design shows that 19 of the 36 reversals hold the selected units fixed, while 17 arise only from changing the selection size and therefore the claim's subject. On compiled Tracr programs the primary test supported 0 of 400 structurally decoy tests and 591 of 760 used-component tests; on 18 trained InterpBench transformers it supported 942 of 942 specification-necessary and 18 of 8858 specification-unnecessary instances, all 18 on nodes the trained networks demonstrably use. On identical measurements with seven injected workflow faults, single-configuration reporting surfaced none; a conventional checker found all seven only with a check written for each, and the audit found all seven from its declared plan. The central findings replicated across five independently trained checkpoints per trained model. BeyondNN does not establish a model's mechanism; it makes the assumptions and counterevidence behind a declared test explicit and checkable.

Index Terms—Interpretability, explainability, faithfulness, causal interventions, attribution, provenance, reproducibility, neural networks.

I. Introduction

INTERPRETABILITY research increasingly communicates through quantitative artifacts. Attribution methods assign relevance to input features [1]–[3], probes measure what can be decoded from hidden states [4], activation patching measures what changes when an internal value is replaced [5], [6], and circuit and dictionary methods propose components and features as the units of a mechanism [7]–[9]. Mature libraries make these artifacts cheap to produce [10]–[14].

An artifact, however, is not the claim that is usually attached to it. Consider a BERT-base sentiment classifier and the held-out sentence "i am sorry that i was unable to get the full brunt of the comedy ." IG [2] ranks the word unable and the separator token [SEP] highest among its 19 positions. That is a statement about one method, one baseline and one target; it does not establish that the two tokens are necessary for the prediction, which is a question about intervention [15], [16]. When we intervene, the answer depends on choices. Replacing both tokens with the [MASK] embedding lowers the class margin from 3.97 by 2.66, more than half the clean margin and more than the drop caused by at least 95% of 50 matched random two-token sets; the predicted class does not change. Eight of the 17 declared alternative configurations contradict this result, and if the claim concerns the two highest-ranked content tokens the selection becomes {unable, get} and the margin drop is negative (Fig. 2).

Conclusions of this kind move when the link between measurement and conclusion changes. Activation-patching results depend on the corruption and the metric [6]; removal-based faithfulness tests depend on how far the replacement is from the data distribution [17], [18]; single-input mediation scores have high variance [19]; subspace interventions can produce illusory localisation [20]; probe accuracy does not show that a model uses what the probe finds [4], [21], [22]; and agreement with human rationales is not faithfulness [15], [23]. Recent position papers argue that interpretability claims should be stated with their estimand, intervention class and identification assumptions [16], [24], and call for systems that make the arguments behind claims auditable [25]. What is missing in practice is an executable link: a representation in which the claim, its units, its replacement, its controls and the alternatives that were tried are explicit enough for a program, not only a reviewer, to check the inference.

We present BeyondNN, an open-source evidence and audit layer for PyTorch [26], [27]. BeyondNN is not an attribution, probing or patching method. It records each measurement as typed, provenance-bound evidence, represents claims as declared objects with a relation, a subject, a scope and an eligibility, and re-derives each claim's standing from the recorded evidence under a plan that assigns every configuration a role (primary, alternative or stress_test). Only primary configurations decide a standing; the others produce typed findings. The audit never runs the model and returns no global score.

The contributions of this paper are:

1) A formal evidence-audit semantics for interpretabil-

N. Roufas and A. Kanavos are with the Department of Informatics, Ionian University, Corfu, Greece (e-mail: inf2024146@ionio.gr; akanavos@ionio.gr).

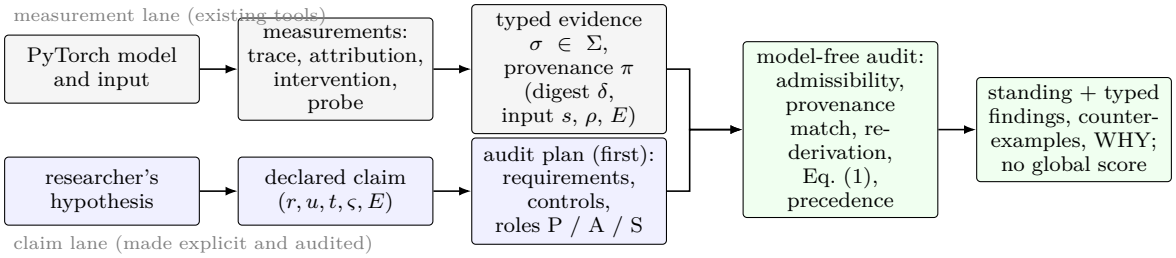


Fig. 1. BeyondNN separates measurements from claims. Measurements become typed, provenance-bound evidence; claims are declared objects; a plan written before the audit states the required evidence, the controls and the role of each configuration. Only primary configurations decide a standing (Proposition 2); the audit re-derives every result from the records and never runs the model.

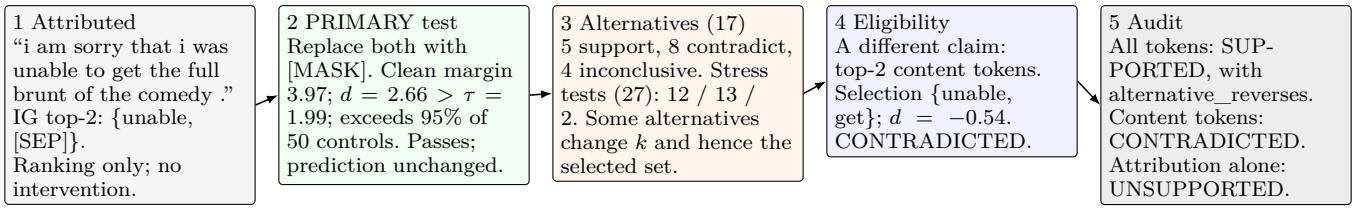


Fig. 2. Running example (BERT-base SST-2, held-out sample 423). Values are read from the per-sample records of the frozen central experiment (central775_D_heldout_{all,content_tokens}.json). supported is scoped to the primary rule and qualified by the alternative finding; the content-token claim concerns a different subject.

ity claims about neural networks: typed evidence, provenance, scoped claims with explicit eligibility, a test outcome rule with an attainability condition, configuration roles, admissibility and a precedence rule for standings, with six properties that hold by construction (Section III).

2) An implementation for PyTorch, released as BeyondNN 0.1.0 (Section IV).
3) An evaluation on trained MLP, CNN, BERT-tiny and BERT-base models and on compiled and SIIT-trained transformers with reference mechanisms (Sections V–VI), including: a fixed-selection decomposition of configuration dependence; a comparison with conventional workflows on identical measurements under injected faults; validation against reference labels that use no intervention on the audited network; replication over independently trained checkpoints; exact re-execution; and the computational cost of the evidence layer.

Throughout, we evaluate claim protocols, not attribution methods. IG is the attribution method in these studies; nothing here shows that IG or attribution in general is unreliable. A supported standing means that the evidence passed the declared primary rule for the stated claim, scope, samples and checkpoint; it is not a statement of logical necessity or of a model's mechanism.

II. Related Work

Attribution and faithfulness: Attribution methods [1]–[3] answer "how relevant is unit u to target T under method M and baseline B". This answer can be insensitive to the model's parameters [38], sensitive to function-preserving transformations [39], and different from attention weights [40], [41]. Jacovi and Goldberg separate plausibility from faithfulness [15]; removal tests such as ERASER comprehensiveness and sufficiency [23] and ROAR [18] operationalise faithfulness and depend on the replacement [17]. In BeyondNN an attribution is attributed evidence and can never decide a causal relation.

Interventions and causal abstraction: Causal tracing and activation patching [5], [6] estimate how an output depends on a component. Causal scrubbing [42] turns a mechanistic hypothesis into a correspondence with a high-level graph tested by resampling ablations and summarised as recovered loss; causal abstraction [43] unifies patching, mediation and scrubbing, and distributed alignment search [44] aligns high-level variables with representations. Causal scrubbing is the closest methodological precedent to BeyondNN's claim–test structure. BeyondNN differs in scope: it records evidence of several epistemic types from several protocols, keeps alternative interventions as named assumptions with roles, and returns a categorical standing with findings rather than a scalar.

Probes and concepts: Decodability does not imply use [4]; control tasks [22] and amnesic probing [21] address this, concept activation vectors [45] measure sensitivity to a direction, and dictionary features [9], [14] are often given generated labels whose text need not reflect the computation [46]. BeyondNN keeps decodability, use and labels apart and derives validated_concept only from passed encoding and use tests.

Evaluation frameworks, benchmarks and statistics: Quantus [47] and OpenXAI [48] score explanation methods; Tracr [36], InterpBench [37], MIB [49], RAVEL [50] and CausalGym [51] provide models or tasks with reference mechanisms. These score methods or proposed mechanisms; BeyondNN audits a particular claim under declared criteria and can itself be evaluated against

TABLE I
Built-in, documented capabilities (current releases checked 2026-11-03). "User": possible with user code on top of the library. The table positions BeyondNN; it does not rank tools that address different problems.

capability	Captum	Quantus	causal scrubbing	NNsight / pyvene / TL	CIF	Certified Circuits	BeyondNN
measurement generation	✓	external	✓	✓	consumes	wraps	partial
explicit claim object (relation, subject, scope)	–	–	partial (hypothesis)	–	partial (estimand)	–	✓
evidence-type admissibility	–	–	n/a	–	n/a	–	✓
checkpoint / input provenance per record	user	user	user	user / partial	user	user	✓
refusal of mismatched provenance or scope	user	user	user	user	user	user	✓
eligibility and estimand matching	–	–	partial	–	✓	–	✓
matched controls as precondition of support	–	baselines	–	–	–	–	✓
declared alternative configurations with roles	–	–	–	–	–	partial	✓
per-sample counterevidence retained	–	scores	aggregate	–	–	abstention	✓
deterministic re-audit of saved evidence	–	–	–	–	–	–	✓
statistical inference / certification	–	–	–	–	✓	✓	–

TABLE II
Evaluation setup. All central studies use correctly classified held-out inputs, one torch thread and CPU execution.

models	MLP (breast cancer [28]; sites input, hidden), CNN (digits [29]; pixels, relu2), BERT-tiny [30], [31] and BERT-base [32], [33] on SST-2 [34], [35] (token embeddings)
held-out samples	51 / 51 / 60 / 60 / 40 / 40 (302)
evidence	G / IG / random top-k, $k \in \{5, 10, 20\}$%; comprehensiveness and sufficiency; 50 count-matched controls; 3 replacements
primary	training mean ([MASK] for BERT), k at 10%, count null at $q = 0.95$, $\tau = 0.5\times$ clean margin
alternative	resample ([PAD]), k at 5% and 20%, magnitude-matched null, 0.5τ
stress_test	zero (MLP input: training minimum), no null, 1.5τ
reference models	8 held-out compiled Tracr programs [36] (40 inputs each); 18 SIIT-trained InterpBench transformers [37] (40 inputs each)
checkpoints	frozen seed-0 MLP and CNN; seeds 0–4 retrained (Linux); pinned Hugging Face revisions for BERT

their references (Section VI-D). Several of these benchmarks define success through interchange interventions, the same family an audit uses, so agreement with them can reflect shared semantics. Méloux et al. [19] show that circuit-discovery estimates are high-variance; Certified Circuits [52] certify the stability of circuit-inclusion decisions under data perturbation; and Certified Interventional Fidelity [53] gives anytime-valid confidence sequences for interventional estimands. BeyondNN has none of these inferential guarantees; it records which estimand and assumptions underlie a standing.

Claims and evidence: Doshi-Velez and Kim ask that evaluation match the claim [54]; Joshi et al. use Pearl's hierarchy to state which claims each kind of evidence can justify [16]; Lin and Liu propose that causal claims disclose their identification strategy and stress-test its assumptions [24]; and Lan et al. call for expert guidelines and source-based auditing of the arguments that interpretability claims depend on [25]. We do not claim that matching claims to evidence, or auditing interpretability arguments, is new. Our contribution is narrower: an implemented, machine-checkable semantics that matches typed evidence to scoped claims by provenance, applies declared primary and alternative rules to saved records, and retains counterevidence. Table I positions it against existing tools.

Tooling: Captum [10], TransformerLens [11], NNsight [12], pyvene [13] and SAELens [14] produce measurements and perform interventions. BeyondNN complements them: it consumes Captum attributions through an adapter, and its compiled-program study is cross-checked against an independent TransformerLens implementation.

III. Formal Framework

This section defines the objects that BeyondNN represents and the rule by which an audit derives a standing from them. The definitions describe the semantics implemented in BeyondNN 0.1.0; each property in Section III-F is stated together with the part of the implementation that enforces it and the test or experiment that checks it. We use one running example: the necessity claim for the Integrated Gradients (IG) selection on held-out sample 423 of a BERT-base sentiment classifier (Fig. 2).

A. Evidence and Provenance

Definition 1 (Evidence record). An evidence record is a tuple $e = (\mathrm{id}, \sigma, v, \pi)$, where v is the recorded value, $\sigma \in \Sigma$ is its epistemic status, π is its provenance, and $\mathrm{id} = H(\sigma, v, \pi, \mathrm{references})$ is a content hash. The status set is $\Sigma = \{$observed, measured, attributed, interventional, estimated_causal, validated_concept, generated$\}$. The status is fixed by the kind of record that produced e and is never supplied by the caller. Statuses are unordered.

Definition 2 (Provenance). The provenance $\pi(e)$ contains the model fingerprint δ (a digest over the names, dtypes, shapes and bytes of all parameters and persistent buffers), the identity $s = H(x)$ of every input, the target metric t, the site, the replacement ρ (name and digest of its value), the declared unit eligibility E, the protocol and its version, and the software versions. Two records have the same conditions if these fields agree; time and process are recorded but are not part of the conditions.

In the running example, the IG relevance of the 19 token positions is an attributed record, and the margin change

$d = 2.66$ obtained by replacing the two selected embeddings with the [MASK] embedding is an interventional record. Both carry the same δ and the same s.

B. Claims, Scope and Eligibility

Definition 3 (Claim). A claim is $c = (r, u, t, \varsigma, E)$ with relation $r \in \{\text{necessary_for, sufficient_for, increases, decreases, attributed_to, encodes}\}$, subject u, target t, estimand scope $\varsigma \in \{\text{instance}, \text{finite_sample}, \text{population}\}$ and eligibility E. The subject is either a declared unit set or a selection rule $\text{sel}_{M,k}(E)$: the k units of the eligible set E ranked highest by method M.

Eligibility is part of the subject. The claims "the IG top-2 tokens" and "the IG top-2 content tokens" differ even on the same input: in the running example the first selects {unable, [SEP]} and the second {unable, get}. Changing k also changes the subject whenever the k-th and (k+1)-th ranked units differ.

C. Tests, Controls and Outcomes

Definition 4 (Necessity test). Let m be the target metric, S the selected units, ρ the replacement, τ the effect threshold, q the control criterion and $S_1, \ldots, S_J$ random control sets matched to S (by count, or by perturbation magnitude) and drawn from E. With $d = m(x) - m(x_{S \leftarrow \rho})$ and d_j the same drop for S_j, the test outcome is

$$o = \begin{cases} \text{inconclusive} & \text{if } x_{S \leftarrow \rho} = x \text{ (no-op)}, \\ \text{supports} & \text{if } d \geq \tau \text{ and } \frac{1}{J} \sum_j \mathbb{1}[d_j < d] \geq q, \\ \text{contradicts} & \text{otherwise.} \end{cases} \tag{1}$$

Control draws identical to S always tie with it and never count as "below". When the largest attainable fraction $\bar{f} = 1 - |\{j : S_j = S\}|/J$ is smaller than q, no model could pass the criterion. The audit then treats a recorded contradicts as uninformative and counts it as inconclusive, with the finding control_criterion_unattainable (ADR-054); the record itself is not changed. The rank criterion q is a threshold on an empirical fraction over J sampled sets; it is not a calibrated p-value. Sufficiency is defined symmetrically by keeping only S.

D. Configurations and Roles

Definition 5 (Configuration and role). The configuration $\kappa(e)$ of a test result is its value on every assumption axis: replacement, k, null (control type and criterion), threshold, method and dataset. A threshold axis value is either the recorded τ or a declared re-evaluation $\alpha\tau$ applied to the recorded d and d_j. An audit plan maps axis values to roles $\{\text{primary} < \text{alternative} < \text{stress_test}\}$. The role of κ is the largest role among its axis values; a configuration with an undeclared value is undeclared and takes no part in the standing.

In the central experiments every sample is tested under 45 configurations: one primary, 17 alternative and 27 stress_test (Table II).

E. Admissibility and Standing

Definition 6 (Admissible evidence). A record e is admissible for claim c in plan P if (i) its status is in the set accepted by the relation, $\sigma(e) \in \text{Adm}(r)$, where causal relations accept only interventional and estimated_causal evidence and attributed_to accepts attributed; (ii) its estimand scope matches ς; (iii) its subject and eligibility match u and E; and (iv) its provenance matches P: $\delta(e)$ equals the declared checkpoint, and every input identity is a declared sample. Records failing (iv) or an integrity check are excluded and the exclusion is reported.

Definition 7 (Standing). For a claim c and a group g (one sample for instance-scope claims), let A be the admissible results and $A_P \subseteq A$ those in primary configurations. The standing is assigned in this order: (1) assumption_sensitive if A_P contains both supports and contradicts and every disagreeing pair differs on a recorded axis, mixed if some pair does not; (2) contradicted if the verdict derived from A_P under the claim's requirement policy is a contradiction; (3) unsupported if a finding marks the evidence as the wrong kind, the required controls as missing, or a declared invariance as untested; (4) supported or inconclusive according to the verdict; (5) not_evaluated if A is empty. alternative and stress_test configurations whose decisive outcome differs from the primary outcome produce the findings alternative_reverses (qualifying) and stress_test_reverses (informational).

A standing is a category, not a confidence level, and the audit computes no global score. For per-sample claims there is no claim-level truth value: the report gives the distribution of standings over the declared samples and the identities of counterexamples.

F. Properties

The following properties hold by construction of the audit rules. They establish consistency with the declared semantics; none of them establishes that a plan, a replacement or a threshold is scientifically appropriate.

Proposition 1 (Admissibility). If no admissible record of c has a status in $\text{Adm}(r)$, the standing of c is neither supported nor contradicted.

Basis. Condition (i) of admissibility filters records before the verdict is derived. Checked by: pre-registered scenarios and code-mutation tests; empirically, every attribution-only audit in this paper (2,480 selection claims) was unsupported (Section VI-B).

Proposition 2 (Role invariance). For a plan with declared roles, the standing of (c, g) is a function of A_P only. Adding, removing or changing alternative or stress_test results can change the findings but never the standing.

Basis. The verdict and the disagreement analysis are computed on primary entries only; other roles enter only the finding generator. Checked by: all 47 held-out samples

with alternative_reverses retain the standing supported (Section VI-A).

Proposition 3 (Monotone reversal count). The number of groups carrying alternative_reverses is non-decreasing in the set of declared alternative configurations.

Basis. The finding is an existential statement over the declared alternatives. Consequently, an at-least-one reversal count measures dependence on a declared test family; it is not an instability rate.

Proposition 4 (Threshold asymmetry). With d, d_j and q fixed, re-evaluating (1) at $\alpha\tau$ with $\alpha < 1$ cannot turn supports into contradicts, and $\alpha > 1$ cannot turn contradicts into supports.

Basis. The outcome is monotone in τ. Checked by: in every re-run reported here, the $\tau \times 0.5$ alternative reversed no supported claim (Table III).

Proposition 5 (Provenance refusal). A result whose model fingerprint or input identity is not declared in the plan never contributes to a standing. A declared group without admissible evidence is not_evaluated.

Basis. Condition (iv) of admissibility. Checked by: the injected wrong-checkpoint and wrong-input faults of Section VI-C (5/5 not_evaluated each).

Proposition 6 (Re-derivability). The audit is a deterministic function of the evidence records, the plan and the audit-semantics version. A stored report can be re-derived from saved evidence without the model, and any edit to a stored record changes its content hash and is rejected on load.

Basis. Content-addressed records and a model-free audit. Checked by: re-audits from disk identical to in-memory reports, and rejection of every tampered trace (Sections VI-C and VI-G).

IV. Implementation

BeyondNN 0.1.0 is a Python library for PyTorch released under the Apache-2.0 licence (pip install beyondnn; source at https://github.com/NikolasRoufas/beyondnn). It has 75 modules, is tested on Python 3.10, 3.12 and 3.14 with PyTorch ≥2.3, and import beyondnn does not import PyTorch. We describe only what the evaluation uses; all names are public API.

Layers: core handles tracing, hooks, sites, sample identity and persistence; schema and provenance hold the immutable, versioned records, statuses, claims and model fingerprints. Five modules produce or assess evidence: attribution, interventions, faithfulness, concepts and audits. explain composes the structured WHY. Every layer below audits produces records; the audit only reads them. Each record kind is versioned and older versions are migrated on load.

Instrumentation: bnn.trace(model, x, sites=...) runs one forward pass and returns a trace; hooks exist only during a recording, and the user's model is never modified. Tracing refuses situations in which a record could be silently wrong (foreign forward hooks, models spread over several devices, structural changes during recording).

Evidence producers: bnn.attribute records attributed evidence (native gradient, input×gradient and IG; Captum methods through an optional adapter, verified with Captum 0.9). bnn.intervene records interventional effects of the declared operations zero, constant, patch and direction on sites, units or feature directions. The faithfulness module implements selection-based tests: F.top_k turns an attribution into a selection (descending absolute score, ties broken by index, optionally among declared eligible units), F.comprehensiveness and F.sufficiency declare the test of (1), F.replacement declares a named replacement tensor (for example a training mean, a resampled activation or a [MASK] embedding), and F.controls declares matched random controls with a recorded seed. There is no implicit zero replacement.

Plans, audits and persistence: An audit plan is itself a content-addressed record. It declares the checkpoint, the samples, the claims, each claim's requirement (assessment policy, whether controls are mandatory, alternative threshold criteria), the invariances the claim asserts, the configuration roles and counterexample caps. bnn.audit(evidence, plan=plan) returns a deterministic report with sorted keys that embeds the plan, a digest of the evidence identifiers and its producer (BeyondNN version and audit-semantics version). AU.save_evidence and AU.load_evidence save and reload exactly the traces an audit ingests, with full re-validation; AU.verify_report(report, evidence, plan) re-derives a stored report and raises an error on any difference. Concept evidence follows the same pattern: an encoding test against random directions and label permutations, and a use test that removes the concept direction and compares the target change with a declared minimum in the target's own scale.

V. Evaluation Design

We ask whether making claims explicit changes what can be concluded, and how the audit rules behave against reference mechanisms and conventional workflows. Seven research questions organise the evaluation: RQ1 How often, and through which assumption, do declared alternatives reverse a primary conclusion? RQ2 Does attribution evidence establish necessity, and is inadmissible evidence refused? RQ3 Which workflow errors does the audit detect, compared with conventional workflows on the same measurements? RQ4 How do primary standings relate to reference labels that are not defined by the audited intervention? RQ5 Does concept validation separate decodability from use? RQ6 Are the central findings stable across independently trained checkpoints? RQ7 Can studies be re-executed and re-audited exactly, and what does the evidence layer cost?

Setup: Table II summarises models, sites, samples and configurations. Central held-out inputs are correctly

classified inputs drawn in seeded order after the development samples, so all central results are conditional on correct prediction. We test necessity and sufficiency of IG, gradient and random top-k selections against the predicted-class margin.

Protocols and blinding: Each original held-out round had a plan frozen before it ran (Phase 7.5 and 7.75 plans in the repository). Later protocol revisions (ADR-053 on eligibility and ADR-054 on attainability) were informed by earlier results, so the work is not uniformly blind: the central 47/52 is a re-audit of held-out samples whose earlier audit had given 49/52. Every experiment new to this paper (Stages 2–4) had a protocol committed before execution, stating hypotheses, units, budgets and stopping rules; one baseline implementation error found during execution is reported in Section VI-C.

Units and statistics: The observational unit is a (site, held-out sample) pair for central claims and a (component, input) instance for reference models. Counts are exact. Wilson and input-level bootstrap intervals are descriptive and unclustered: instances share components and programs, and samples share checkpoints. Between-checkpoint spread is reported directly (RQ6). No interval here quantifies uncertainty over new models.

VI. Results

A. RQ1: Configuration Dependence, Decomposed

On the six central sites, IG necessity was primary-supported on 52 of 302 held-out samples, and on 47 of these at least one declared alternative configuration contradicted the primary result. The committed per-sample records store only role-by-outcome counts, and their recorded reversal axes merge alternative and stress_test findings, so they cannot attribute reversals to assumptions. We therefore re-ran the frozen central design unchanged, exporting every tested configuration with its axis values. Using the frozen macOS checkpoints on a Linux x86-64 server, the re-run reproduced every per-sample standing, finding code and outcome profile of the frozen results (0 differences over all six sites and six claims).

Table III and Fig. 3 decompose the reversals. A configuration that keeps k keeps the selected units, because the IG ranking does not depend on the replacement, null or threshold; a configuration that changes k changes the claim's subject. Of the 36 reversed samples, 19 were reversed by an alternative that keeps the selected units fixed, and 17 only by alternatives that change k. Changing only the replacement reversed 15 samples and changing only the null (count- to magnitude-matched controls) 11; no reversal required a combination of axes without a single-axis explanation. The halved threshold reversed none of the supported claims, as Proposition 4 requires. The split differs by model: on the CNN pixel site 10 of 13 reversals hold the units fixed, whereas on BERT-tiny only 3 of 13 do and 10 arise only from a change of k. All reversed samples retain the standing supported with the qualifying finding alternative_reverses (Proposition 2).

TABLE III
Decomposition of ALTERNATIVE reversals of PRIMARY-supported IG necessity claims (re-run of the frozen central design, held-out samples). Any: ≥ 1 ALTERNATIVE configuration contradicts. Fixed: a contradicting ALTERNATIVE keeps k, hence the selected units. Single-axis columns: contradiction by a configuration that differs from PRIMARY only on that axis. k only: reversed only by configurations that change k. Counts per sample; samples are nested in four checkpoints.

			reversed		single-axis reversal				
site	n	sup.	any	fixed	repl.	null	thr.	k	k only
MLP input	51	8	7	3	3	1	0	5	4
MLP hidden	51	4	2	2	2	0	0	2	0
CNN pixels	60	14	13	10	9	7	0	7	3
CNN relu2	60	1	1	1	1	0	0	1	0
BERT-tiny	40	14	13	3	0	3	0	11	10
Total	262	41	36	19	15	11	0	26	17

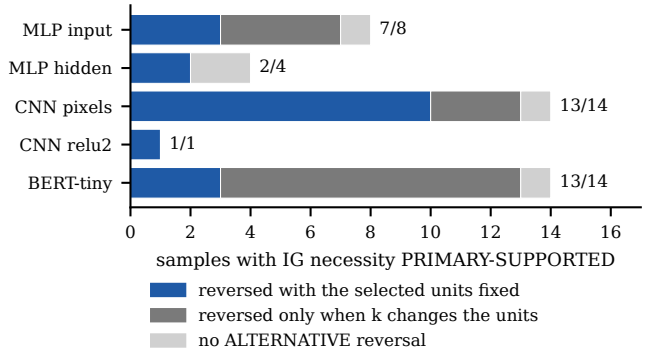


Fig. 3. ALTERNATIVE reversals of PRIMARY-supported IG necessity claims by source (re-run of the frozen design; Table III). Counts per held-out sample.

These counts are conditional on the declared family (Proposition 3) and are not instability rates. They separate two readings that the aggregate 47/52 conflated: dependence of a fixed claim on the intervention or control rule, and dependence of the claim itself on the selection size.

B. RQ2: Attribution, Necessity and Admissibility

On BERT-base, IG's primary selection contained [SEP] or [CLS] on 30 of 40 samples, and [SEP] on 10 of the 11 samples where necessity was supported. Declaring the claim the analyst usually intends, about the top-k content tokens with k, rankings and controls computed over content tokens only, gave 6 of 40 supports [0.07, 0.29]. Only 3 of the 11 all-token supports survived; 5 became contradicted and 3 inconclusive (a single content token leaves the control criterion unattainable), and 3 samples became newly supported. The special-token pattern was known from an earlier round before the content-token claim was declared. By the admissibility rule, attribution-only evidence never supported an intervention claim: all 2,480 attribution-only selection claims in this work (280 central, 1,960 InterpBench, 240 Tracr) were unsupported with

TABLE IV
Injected workflow faults on identical measurements (MLP, 20 held-out samples). Detected/affected, false alerts in parentheses. B0: single-configuration report. B1: conventional log with 54 lines of fault-specific checks. BeyondNN: the frozen plan, no fault-specific code. *Study-level refusal: the edited trace fails its content hash and the evidence set is not loaded.

fault	aff.	wrong sup.	B0	B1	BeyondNN
F1 attribution as necessity	20	20	0	20/20 (0)	20/20 (0)
F2 wrong checkpoint	5	1	0	5/5 (0)	5/5 (0)
F3 wrong input	5	0	0	5/5 (0)	5/5 (0)
F4 wrong selection size	5	1	0	5/5 (0)	5/5 (0)
F5 omitted controls	20	6	0	20/20 (0)	20/20 (0)
F6 configuration dependence	4	4	0	4/4 (1)	4/4 (0)
F7 tampered outcome	5	5	0	5/5 (0)	5/5* (–)

attribution_is_not_intervention (Proposition 1). This is a consequence of the type rule, not empirical evidence that attribution is unreliable.

C. RQ3: Comparison with Conventional Workflows

To test what the audit adds, we compared it with conventional workflows on identical measurements: the frozen central evidence for 20 held-out MLP samples, logged as a careful researcher would (checkpoint and input hashes, configuration, drops, control drops, outcome). B0 reports the primary outcome; B1 adds 54 lines of checks written for this study (checkpoint, input, intervention present, controls present, k, outcome recomputation, and a multiverse rule); BeyondNN audits with the frozen plan (98 lines, reused unchanged). We injected seven faults into both representations (Table IV). A first run used a baseline that ranked IG scores by signed rather than absolute value; after correcting it to the library's rule, the baseline and BeyondNN selected identical units and reached identical outcomes on 20 of 20 samples (largest drop difference 2.9×10^{-6}).

Single-configuration reporting surfaced none of the faults and would have reported 37 erroneous supported conclusions. The careful checker detected every fault, but only through a check written for that fault. The audit detected every fault from its declared plan without fault-specific code and with no false alert on unaffected samples. The two differed in two respects. B1's multiverse rule flagged one additional sample whose only contradicting alternative had an unattainable control criterion; the flat log records it as a contradiction, whereas the audit treats it as uninformative. And B1 localised tampered outcomes to the sample, whereas BeyondNN refused the whole evidence set because the edited record no longer matched its content hash. The result is therefore not that the audit detects errors a careful researcher cannot, but that its checks are part of the representation, apply to any plan, and encode edge cases (attainability, eligibility, provenance) that hand-written checks must anticipate. By lines of code it was not cheaper in this study.

D. RQ4: Reference Labels Not Defined by the Audited Intervention

Compiled programs: Eight held-out Tracr programs [36] label each component by whether its variable is read by the output expression, independently of any intervention; the labels describe structural use, not per-input necessity. The primary resample test supported 0 of 400 decoy component–input tests (95% Wilson upper bound 0.95%), including 0 of 57 IG-selected decoy heads, and 591 of 760 tests of used components (77.8% [74.7, 80.6]); each of the 19 used components passed on 28–38 of its 40 inputs and each of the 10 decoys on none. Two programs contain decoys correlated with the used variable: 0 of their 160 decoy tests were supported, of which 80 test an exact copy of the used variable. The 169 contradicted tests of used components are inputs on which the sampled counterfactual left the relevant value unchanged; an independent TransformerLens interchange agreed with the primary outcome on all 1,160 decisive instances. Mean replacement behaved similarly (0/400; 586/760), whereas zero replacement, a stress_test, met the support rule for all 400 decoy and 760 used tests, because zero is not a valid one-hot value (Fig. 4a).

Trained transformers: InterpBench's 18 held-out SIIT-trained transformers [37] were previously scored against labels filtered by the trained models' own resample ablations, which makes agreement partly circular. We re-scored the committed standings against specification labels that use no intervention on the trained network: a node–input instance is necessary if the node is in the benchmark circuit and interchanging the corresponding node of the compiled high-level program changes that program's output. The primary test supported 942 of 942 specification-necessary instances and 18 of 8858 specification-unnecessary instances (0.20%); zero replacement supported 1155 (13.0%) of the latter (Fig. 4b). All 18 disagreements are non-circuit nodes, 17 of them in one model, on which the trained network's own resample ablation flips its prediction: the network does not implement its specification exactly there, and the audit reports the network, not the specification. This re-analysis was performed after the earlier results were known, and SIIT training itself uses interchange interventions, so the labels are independent of the audited evaluation but not of the intervention family that shaped the networks.

E. RQ5: Decodability versus Use

Four held-out numeric Tracr programs each contain a used indicator (K+), a decoy indicator read with coefficient exactly zero (K−) and a permuted-label control (K0). The encoding test supported K− in all four programs, so decodability did not separate the variables; the use test did. Removing the K+ direction changed the target by 0.16–0.30, above the declared scale-relative minimum (0.2 standard deviations of the training target, ADR-055), and removing K− changed it by exactly zero. All 4 K+ concepts were validated and none of the 8 K−

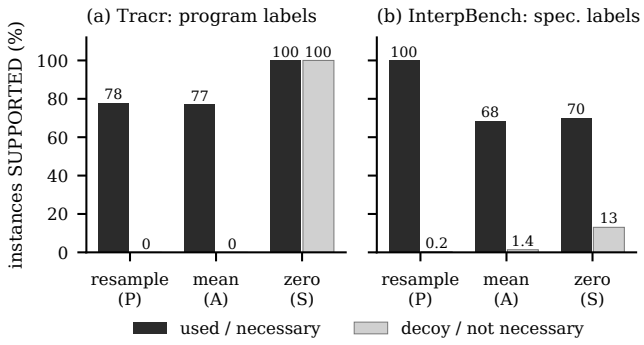


Fig. 4. Each configuration evaluated as if it were the only one, against reference labels not defined by the audited intervention: (a) program-defined structural labels of 8 compiled Tracr programs; (b) specification labels of 18 SIIT-trained InterpBench transformers. Percentages of instances supported; unclustered.

or K0 concepts under either threshold rule. The held-out programs did not discriminate between the scale-relative and an absolute threshold; on a constructed grid (6 use weights $\times$ 5 output scales) the absolute rule changed its verdict with the output scale in both directions while the relative rule validated exactly the weights ≥ 0.5 at every scale. The relative rule was adopted after an absolute rule had failed to validate a known-used Tracr variable (use effect 0.06 against 0.1); no concept in a trained network has independent truth here.

F. RQ6: Independently Trained Checkpoints

All central findings above rest on one checkpoint per model. We retrained the MLP and CNN with five seeds and re-ran the full central design on each checkpoint's held-out samples (Table V, Fig. 5). Two hypotheses stated before the runs held on every checkpoint and site: IG necessity support exceeded random-selection support, and wherever at least three IG claims were supported, at least half carried an alternative reversal (75–100%). The fixed-subject share replicated on the CNN pixel site (10–15 of 12–22 supports, 60–83%). Absolute counts varied: IG supports on CNN pixels ranged from 12 to 22 of 60, so a single-checkpoint count is not a stable model-class estimate. On the CNN relu2 site, every IG necessity test carried control_criterion_unattainable on every checkpoint, so its few supports come from the remaining decisive tests. Gradient top-k was supported on 37–41 of 60 CNN pixel samples on every checkpoint, against 12–22 for IG. The same code and seed produced different checkpoint digests on Linux and macOS; on these checkpoints the central counts were nevertheless identical. The BERT models have one checkpoint each.

G. RQ7: Re-execution, Re-audit and Cost

Re-execution: Re-running the frozen central design from source on a different platform reproduced every per-sample audit output of the four central models (RQ1). For the MLP, evidence saved to disk (1.7 and 2.0 GB per site), reloaded in the same run and re-audited gave reports identical to the in-memory ones, and verify_report re-derived each saved report from disk. Every tampered trace was refused on load (RQ3).

Cost: Table VI and Fig. 6 separate evidence generation from the model-free audit. The audit and its verification are independent of the model, so their share falls as models grow: about a third of the total for the MLP and CNN sites and a few percent for the BERT models. At the level of one test, the evidence layer is not free: one primary IG necessity test with 50 controls on the 30-feature MLP took 55.3 ms through BeyondNN and 2.38 ms in plain PyTorch with Captum, about 23 times longer, because record construction, hashing and provenance dominate the computation of so small a model. Saved evidence is large when every control effect is retained (33–39 MB per MLP sample).

TABLE V
Independently trained checkpoints (Linux x86-64; seeds 0–4) and the frozen macOS checkpoint. Each cell: IG necessity PRIMARY-supported / of which ALTERNATIVE-reversed / random selection supported, out of 51 (MLP) or 60 (CNN) held-out samples drawn by the same rule per checkpoint.

site	frozen	seed 0	seed 1	seed 2	seed 3	seed 4
MLP input	8/7/1	8/7/1	8/8/0	7/6/0	4/4/1	7/6/1
MLP hidden	4/2/0	4/2/0	2/1/0	2/2/0	2/2/0	2/2/0
CNN pixels	14/13/1	14/13/1	22/20/2	20/19/3	17/16/0	12/12/1
CNN relu2	1/1/0	1/1/0	2/1/1	3/3/1	4/3/0	4/3/0

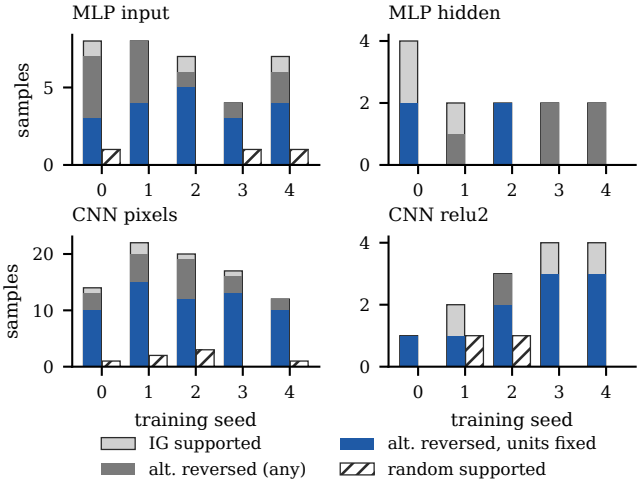


Fig. 5. Central findings on five independently trained checkpoints per model (Table V). Held-out sets are drawn by the same rule per checkpoint and differ between checkpoints.

H. Case Study: Plausibility versus Necessity on e-SNLI

On 40 held-out e-SNLI samples [55], [56], with BERT-base fine-tuned on SNLI, we compared three equal-size token selections: annotator 1's rationale, IG's top-k and a random set, each tested for necessity with the [MASK] replacement and matched controls. IG overlapped more with the human rationale than random sets did (token

TABLE VI
Cost per held-out sample of the full central design (one CPU thread, Linux x86-64). Evidence: model execution and BeyondNN record construction for all results of the sample. Audit and verify: model-free. Share: audit + verify as % of the total. Report: size of the site's report (MB). RSS: peak resident memory of one process (MB).

site	n	res.	evid. (s)	audit (s)	verify (s)	share	report	RSS
MLP input	51	72	3.7	0.967	1.154	36.7	20.3	5906
MLP hidden	51	72	4.4	1.033	1.206	33.7	20.0	6866
CNN pixels	60	72	3.7	1.132	1.181	38.8	23.9	3364
CNN relu2	60	72	4.3	1.099	1.025	33.2	23.6	3836
BERT-tiny	40	69	70.4	1.211	1.105	3.2	15.4	1427

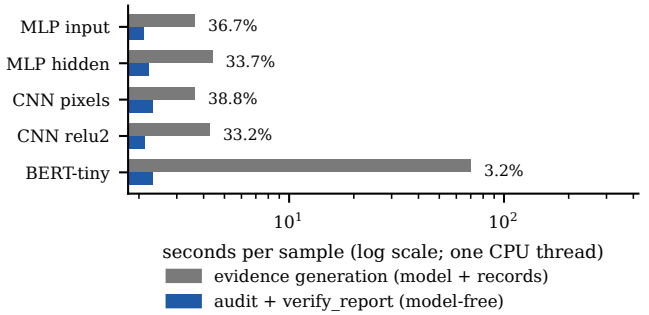


Fig. 6. Cost per held-out sample of the full central design: evidence generation (model execution and record construction) versus the model-free audit and verification; labels give the audit share. One CPU thread, Linux x86-64.

F1 0.34 [0.26, 0.42]; IG minus random +0.149, Bonferroni 98.75% [0.031, 0.270]), yet necessity was supported for the human rationale on 10, for IG on 7 and for random sets on 3 of 40 samples, opposite to the directional prediction; the paired IG-minus-human difference was −0.075 [−0.225, 0.075], so no population difference is established. On 17 of the 40 samples the model predicts the same label with an empty premise (empty-premise accuracy 0.456 against chance 1/3 [57]), and 5 of IG's 7 supports fall in that stratum (exploratory). Human rationales are plausibility references, not causal ground truth; one annotator, one model and 40 samples limit this study.

VII. Limitations

What a standing means: supported means that the primary evidence met the declared rule for the stated claim, scope, samples and checkpoint; it is neither a universal truth value nor a population claim, and unsupported and inconclusive are refusals, not refutations. The properties of Section III-F establish consistency with the implemented semantics. They do not establish that a plan, replacement, threshold or control is scientifically adequate, and evidence produced by a flawed protocol is re-derived faithfully by the same code. A plan cannot reveal alternatives that were never declared, and the at-least-one reversal count grows with the declared family (Proposition 3).

Reference truth: Program-defined labels exist only for small compiled programs (one or two layers, at most three heads, one-hot or numeric variables), and they mark structural use, not per-input necessity. The specification labels of the SIIT-trained InterpBench models use no intervention on the trained network, but SIIT training itself uses interchange interventions, so the labels are independent of the audited evaluation and not of the intervention family that shaped the networks. No central model (MLP, CNN, BERT) has independent mechanism truth, and the InterpBench re-analysis was performed after the Phase-7.5 results were known.

Protocol dependence and blinding: Researchers choose the primary configuration, alternatives, thresholds and controls. Each held-out round had a frozen plan, but plans were revised after earlier results (ADR-053/054); the reported 47/52 is a re-audit of samples whose earlier audit (49/52) had been seen. The 169 contradicted tests of structurally used Tracr components are single-counterfactual non-changes, not false negatives against per-input labels; multiple counterfactual draws would address this at higher evidence cost.

Statistics and replication: Wilson and bootstrap intervals are descriptive and unclustered: instances share programs, inputs share checkpoints. The MLP and CNN results were replicated over five independently trained checkpoints; BERT-tiny and BERT-base each have one checkpoint. Nothing here supports generalisation to new architectures or scales; the largest model is BERT-base (110M parameters) on CPU.

Comparison and cost: The comparative study uses faults designed by the authors and a conventional checker written knowing them; it measures which checks exist by default, not how often researchers err. On a 30-feature MLP the evidence layer made one test about 23 times slower than plain PyTorch; its share of total cost shrinks with model size (Table VI), but for very small models it dominates. Hooks observe only instrumented module boundaries, and model fingerprints cover parameters and persistent buffers, not Python source or undeclared constructor settings.

VIII. Conclusion

Between an interpretability measurement and the conclusion drawn from it lies a claim with a relation, a scope, a set of units, a replacement, a control and a threshold. BeyondNN makes that claim the object of analysis: evidence carries its epistemic status and provenance, claims are declared objects, configurations have declared roles, and audits re-derive every standing from recorded evidence without a global score. On trained MLP, CNN and BERT models, most primary-supported IG necessity claims carried a qualifying reversal; with the selected units held fixed, the replacement and the control rule alone reversed many of them, while on the token models most reversals came only from changes of the selection size, that is, of the claim's subject. Against program-defined and specification labels, the primary test supported no

decoy and almost no specification-excluded node, and the exceptions were nodes the trained network demonstrably uses. On identical measurements, the audit surfaced every injected workflow fault without fault-specific code, which a careful conventional checker matched only with checks written for each fault. BeyondNN does not determine what a model computes and cannot supply evidence that was never recorded; it makes interpretability claims explicit, testable, provenance-aware and reproducible, and makes claims that the evidence does not support easier to reject.

Code and Data Availability

BeyondNN 0.1.0, all experiment scripts, frozen plans and committed per-sample results are available at https://github.com/NikolasRoufas/beyondnn; the revision experiments of this paper are in experiments/tnnls_revision/, with protocols in docs/tnnls_revision/protocols/ and a claim-to-source table in docs/tnnls_revision/RESULT_SOURCE_TRACEABILITY.csv.

Acknowledgment

[AI-assistance disclosure for author review, following the IEEE Author Guidelines for AI-generated text.] An AI coding assistant (Claude Code, Anthropic) was used in the revision of this article: to write analysis and experiment scripts (Section VI; repository directory experiments/tnnls_revision/), to recompute reported values from committed artifacts, and to draft and edit the text of all sections and the figure and table generation code. All experiments, numbers and references were checked against the repository's artifacts; the authors reviewed and take responsibility for the content.

References

[1] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep inside convolutional networks: Visualising image classification models and saliency maps," in Int. Conf. Learning Representations (ICLR), Workshop Track, 2014.

[2] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in Proc. 34th Int. Conf. Machine Learning (ICML), ser. PMLR, vol. 70, 2017.

[3] S. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[4] Y. Belinkov, "Probing classifiers: Promises, shortcomings, and advances," Computational Linguistics, vol. 48, no. 1, pp. 207–219, 2022.

[5] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[6] F. Zhang and N. Nanda, "Towards best practices of activation patching in language models: Metrics and methods," in Int. Conf. Learning Representations (ICLR), 2024.

[7] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, "Interpretability in the wild: A circuit for indirect object identification in GPT-2 small," in Int. Conf. Learning Representations (ICLR), 2023.

[8] A. Conmy, A. N. Mavor-Parker, A. Lynch, S. Heimersheim, and A. Garriga-Alonso, "Towards automated circuit discovery for mechanistic interpretability," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[9] R. Huben, H. Cunningham, L. R. Smith, A. Ewart, and L. Sharkey, "Sparse autoencoders find highly interpretable features in language models," in Int. Conf. Learning Representations (ICLR), 2024.

[10] N. Kokhlikyan, V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, and O. Reblitz-Richardson, "Captum: A unified and generic model interpretability library for PyTorch," arXiv:2009.07896, 2020.

[11] N. Nanda and J. Bloom, "TransformerLens," https://github.com/TransformerLensOrg/TransformerLens, 2022.

[12] J. Fiotto-Kaufman, A. R. Loftus, E. Todd, J. Brinkmann, K. Pal, D. Troitskii, M. Ripa, A. Belfki, C. Rager, C. Juang, A. Mueller, S. Marks, A. S. Sharma, F. Lucchetti, N. Prakash, C. Brodley, A. Guha, J. Bell, B. C. Wallace, and D. Bau, "NNsight and NDIF: Democratizing access to open-weight foundation model internals," in Int. Conf. Learning Representations (ICLR), 2025.

[13] Z. Wu, A. Geiger, A. Arora, J. Huang, Z. Wang, N. D. Goodman, C. D. Manning, and C. Potts, "pyvene: A library for understanding and improving PyTorch models via interventions," in Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Human Language Technologies (Volume 3: System Demonstrations), 2024, pp. 158–165.

[14] J. Bloom, C. Tigges, A. Duong, and D. Chanin, "SAELens," https://github.com/decoderesearch/SAELens, 2024.

[15] A. Jacovi and Y. Goldberg, "Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness?" in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics, 2020, pp. 4198–4205.

[16] S. Joshi, A. Mueller, D. Klindt, W. Brendel, P. Reizinger, and D. Sridhar, "Position: Causality is key for interpretability claims to generalise," in Proc. 43rd Int. Conf. Machine Learning (ICML), 2026, arXiv:2602.16698.

[17] P. Hase, H. Xie, and M. Bansal, "The out-of-distribution problem in explainability and search methods for feature importance explanations," in Advances in Neural Information Processing Systems (NeurIPS), 2021.

[18] S. Hooker, D. Erhan, P.-J. Kindermans, and B. Kim, "A benchmark for interpretability methods in deep neural networks," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[19] M. Méloux, F. Portet, and M. Peyrard, "Mechanistic interpretability as statistical estimation: A variance analysis," in Proc. 43rd Int. Conf. Machine Learning (ICML), 2026, arXiv:2510.00845.

[20] A. Makelov, G. Lange, A. Geiger, and N. Nanda, "Is this the subspace you are looking for? An interpretability illusion for subspace activation patching," in Int. Conf. Learning Representations (ICLR), 2024.

[21] Y. Elazar, S. Ravfogel, A. Jacovi, and Y. Goldberg, "Amnesic probing: Behavioral explanation with amnesic counterfactuals," Trans. Assoc. Comput. Linguistics, vol. 9, pp. 160–175, 2021.

[22] J. Hewitt and P. Liang, "Designing and interpreting probes with control tasks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing, 2019, pp. 2733–2743.

[23] J. DeYoung, S. Jain, N. F. Rajani, E. Lehman, C. Xiong, R. Socher, and B. C. Wallace, "ERASER: A benchmark to evaluate rationalized NLP models," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics, 2020, pp. 4443–4458.

[24] Z. Lin and F. Liu, "Position: Mechanistic interpretability must disclose identification assumptions for causal claims," arXiv:2605.08012, 2026.

[25] M. Lan, N. F. Oozeer, C. Bandi, P. Quirke, A. Meek, F. Barez, and A. Abdullah, "Make mechanistic interpretability auditable: A call to develop guidelines via continuous collaborative reviewing," in Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers), 2026, pp. 3516–3540.

[26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[27] N. Roufas and A. Kanavos, "BeyondNN: An interpretability evidence framework for PyTorch, version 0.1.0," https://github.com/NikolasRoufas/beyondnn, 2026.

[28] W. Wolberg and O. Mangasarian, "Breast cancer Wisconsin (diagnostic)," UCI Machine Learning Repository, 1993.

[29] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits," UCI Machine Learning Repository, 1998.
[30] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," arXiv:1908.08962, 2019.
[31] E. Frantar, E. Kurtic, and D. Alistarh, "M-FAC: Efficient matrix-free approximations of second-order information," in Advances in Neural Information Processing Systems (NeurIPS), 2021.
[32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Human Language Technologies, 2019, pp. 4171–4186.
[33] J. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations, 2020, pp. 119–126.
[34] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in Proc. Conf. Empirical Methods in Natural Language Processing, 2013, pp. 1631–1642.
[35] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in Proc. EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP, 2018, pp. 353–355.
[36] D. Lindner, J. Kramár, S. Farquhar, M. Rahtz, T. McGrath, and V. Mikulik, "Tracr: Compiled transformers as a laboratory for interpretability," in Advances in Neural Information Processing Systems (NeurIPS), 2023.
[37] R. Gupta, I. Arcuschin, T. Kwa, and A. Garriga-Alonso, "InterpBench: Semi-synthetic transformers for evaluating mechanistic interpretability techniques," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2024.
[38] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in Advances in Neural Information Processing Systems (NeurIPS), 2018.
[39] P.-J. Kindermans, S. Hooker, J. Adebayo, M. Alber, K. T. Schütt, S. Dähne, D. Erhan, and B. Kim, "The (un)reliability of saliency methods," in Explainable AI: Interpreting, Explaining and Visualizing Deep Learning, ser. Lecture Notes in Computer Science. Springer, 2019, vol. 11700, pp. 267–280.
[40] S. Jain and B. C. Wallace, "Attention is not explanation," in Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Human Language Technologies, 2019, pp. 3543–3556.
[41] S. Wiegreffe and Y. Pinter, "Attention is not not explanation," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing, 2019, pp. 11–20.
[42] L. Chan, A. Garriga-Alonso, N. Goldowsky-Dill, R. Greenblatt, J. Nitishinskaya, A. Radhakrishnan, B. Shlegeris, and N. Thomas, "Causal scrubbing: A method for rigorously testing interpretability hypotheses," AI Alignment Forum (Redwood Research), https://www.alignmentforum.org/posts/JvZhhzycHu2Yd57RN, 2022.
[43] A. Geiger, D. Ibeling, A. Zur, M. Chaudhary, S. Chauhan, J. Huang, A. Arora, Z. Wu, N. Goodman, C. Potts, and T. Icard, "Causal abstraction: A theoretical foundation for mechanistic interpretability," J. Mach. Learn. Res., vol. 26, no. 83, pp. 1–64, 2025.
[44] A. Geiger, Z. Wu, C. Potts, T. Icard, and N. D. Goodman, "Finding alignments between interpretable causal variables and distributed neural representations," in Proc. Third Conf. Causal Learning and Reasoning (CLeaR), ser. PMLR, vol. 236, 2024, pp. 160–187.
[45] B. Kim, M. Wattenberg, J. Gilmer, C. Cai, J. Wexler, F. Viegas, and R. Sayres, "Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (TCAV)," in Proc. 35th Int. Conf. Machine Learning (ICML), ser. PMLR, vol. 80, 2018.
[46] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in Advances in Neural Information Processing Systems (NeurIPS), 2023.
[47] A. Hedström, L. Weber, D. Krakowczyk, D. Bareeva, F. Motzkus, W. Samek, S. Lapuschkin, and M. M.-C. Höhne, "Quantus: An explainable AI toolkit for responsible evaluation of neural network explanations and beyond," J. Mach. Learn. Res., vol. 24, no. 34, pp. 1–11, 2023.
[48] C. Agarwal, S. Krishna, E. Saxena, M. Pawelczyk, N. Johnson, I. Puri, M. Zitnik, and H. Lakkaraju, "OpenXAI: Towards a transparent evaluation of model explanations," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, vol. 35, 2022, pp. 15 784–15 799.
[49] A. Mueller, A. Geiger, S. Wiegreffe, D. Arad, I. Arcuschin, A. Belfki, Y. S. Chan, J. Fiotto-Kaufman, T. Haklay, M. Hanna, J. Huang, R. Gupta, Y. Nikankin, H. Orgad, N. Prakash, A. Reusch, A. Sankaranarayanan, S. Shao, A. Stolfo, M. Tutek, A. Zur, D. Bau, and Y. Belinkov, "MIB: A mechanistic interpretability benchmark," in Proc. 42nd Int. Conf. Machine Learning (ICML), ser. PMLR, vol. 267, 2025, pp. 45 069–45 108.
[50] J. Huang, Z. Wu, C. Potts, M. Geva, and A. Geiger, "RAVEL: Evaluating interpretability methods on disentangling language model representations," in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers), 2024, pp. 8669–8687.
[51] A. Arora, D. Jurafsky, and C. Potts, "CausalGym: Benchmarking causal interpretability methods on linguistic tasks," in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers), 2024, pp. 14 638–14 663.
[52] A. Anani, T. Lorenz, B. Schiele, M. Fritz, and J. Fischer, "Certified circuits: Stability guarantees for mechanistic circuits," arXiv:2602.22968, 2026.
[53] A. Asiaee, "Certified interventional fidelity: Anytime-valid, adaptive evaluation of causal claims in mechanistic interpretability," in Proc. 42nd Conf. Uncertainty in Artificial Intelligence, ser. PMLR, vol. 337, 2026, pp. 210–227.
[54] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv:1702.08608, 2017.
[55] O.-M. Camburu, T. Rocktäschel, T. Lukasiewicz, and P. Blunsom, "e-SNLI: Natural language inference with natural language explanations," in Advances in Neural Information Processing Systems (NeurIPS), 2018.
[56] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in Proc. Conf. Empirical Methods in Natural Language Processing, 2015, pp. 632–642.
[57] S. Gururangan, S. Swayamdipta, O. Levy, R. Schwartz, S. R. Bowman, and N. A. Smith, "Annotation artifacts in natural language inference data," in Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Human Language Technologies, Volume 2 (Short Papers), 2018, pp. 107–112.